IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 4 September 2024, accepted 2 December 2024, date of publication 9 December 2024, date of current version 17 December 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3513155

RESEARCH ARTICLE

QuIM-RAG: Advancing Retrieval-Augmented Generation With Inverted Question Matching for Enhanced QA Performance

BINITA SAHA, UTSHA SAHA, (Graduate Student Member, IEEE), AND MUHAMMAD ZUBAIR MALIK

North Dakota State University, Fargo, ND 58108, USA

Corresponding author: Binita Saha (binita.saha@ndsu.edu)

This work was supported by the Center for Computationally Assisted Science and Technology (CCAST) at North Dakota State University, funded by NSF MRI under Award 2019077.

ABSTRACT This work presents a novel architecture for building Retrieval-Augmented Generation (RAG) systems to improve Question Answering (QA) tasks from a target corpus. Large Language Models (LLMs) have revolutionized the analyzing and generation of human-like text. These models rely on pre-trained data and lack real-time updates unless integrated with live data tools. RAG enhances LLMs by integrating online resources and databases to generate contextually appropriate responses. However, traditional RAG still encounters challenges like information dilution and hallucinations when handling vast amounts of data. Our approach addresses these challenges by converting corpora into a domain-specific dataset and RAG architecture is constructed to generate responses from the target document. We introduce QuIM-RAG (Question-to-question Inverted Index Matching), a novel approach for the retrieval mechanism in our system. This strategy generates potential questions from document chunks and matches these with user queries to identify the most relevant text chunks for generating accurate answers. We have implemented our RAG system on top of the open-source Meta-LLaMA3-8B-instruct model by Meta Inc. that is available on Hugging Face. We constructed a custom corpus of 500+ pages from a high-traffic website accessed thousands of times daily for answering complex questions, along with manually prepared ground truth QA for evaluation. We compared our approach with traditional RAG models using BERT-Score and RAGAS, state-of-the-art metrics for evaluating LLM applications. Our evaluation demonstrates that our approach outperforms traditional RAG architectures on both metrics.

INDEX TERMS Large language models (LLMs), retrieval-augmented generation (RAG), question answering (QA), ChatGPT, GPT-3.5-turbo, Meta-LLaMA3-8B-instruct, hallucination mitigation, information dilution.

I. INTRODUCTION

Large Language Models (LLMs) have revolutionized the field of Natural Language Processing (NLP) and fundamentally changed the way we used to interact with digital information. They have the capability to analyze and generate text that resembles human language, which has become integral across various sectors [1]. However, when it comes to answering domain-specific questions where accuracy and reliability are key, the effectiveness of these models faces challenges due to knowledge limiation, contextual misinterpretation, task-specific variability, etc. [2], [3].

To address these limitations of adapting LLMs for specialized domains, two principal strategies have emerged: a) fine-tuning the models with domain-specific data, and b) augmenting them with external knowledge. Fine-tuning involves additional training of a pre-existing LLM using specialized data, which optimizes the model for particular tasks and significantly boosts its performance [1]. Despite its effectiveness, fine-tuning is not without drawbacks; it is computationally demanding, costly, and runs the risk of forgetting critical information—where the model loses its

The associate editor coordinating the review of this manuscript and approving it for publication was Amin Zehtabian.

© 2024 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

ability to recall previously learned information. [4], [5] [6], [7]. Moreover, the effectiveness of fine-tuning is dependent on the availability of extensive relevant data, which makes it less practical for domain-specific fields.

Another approach to overcome the challenges of customizing LLMs for domain-specific tasks and enhancing precision in generating Q&A responses is the use of Retrieval-Augmented-Generation (RAG) [8]. The foundation of RAG is its capability to incorporate relevant information from external sources to ensure the generated responses are contextually appropriate [9], [10]. This method allows LLMs to adapt to domain-specific tasks more flexibly, sidestepping the high costs and potential knowledge loss associated with fine-tuning. However, RAG faces two significant challenges: information dilution and hallucination [11]. Information dilution arises when the volume of data is so large that it compromises the specificity and accuracy of the responses. Hallucination, on the other hand, refers to instances when the model produces outputs that are linguistically coherent but factually inaccurate or irrelevant to the input query [12].

RAG can effectively handle domain-specific question answering using vectorization. Initially, the dataset is stored in a vector database using an embedding model. Embedding models transform text into numerical representations, or vectors, by capturing semantic meanings and relationships between words, which will allow for efficient similarity search later. When a user submits a query, it also converts into a query vector using the same embedding model. This ensures that both the dataset and the query are represented in comparable vector space. The retrieval component then matches the query vector against the database vector using similarity measures such as cosine similarity. The actual text associated with the closest vectors is retrieved using the retrieval component of RAG. Then LLm uses this retrieved text as context to generate a response.

This paper makes two key contributions: a) preparing domain-specific dataset to enhance the performance of RAG system b) The development of a novel modified RAG system, named QuIM-RAG (Question-to-question Inverted Index Matching), which introduces an inverted question matching approach with a quantized embedding index to improve the precision and efficiency of the information retrieval and answer generation processes. The domain-specific dataset is used to enhance the performance of RAG with a focus on data quality, relevance, and verifiable sources. We develope a comprehensive methodological approach for the data preparation phase, which included systematic data collection, cleaning, and structured organization. This approach is designed to improve the quality and reliability of the dataset corpus. The goal is to convert unprocessed data collected from websites into manageable chunks, which are then transformed into potential questions with the help of gpt-3.5-turbo-instruct model. A proper custom prompt is used to ensure that all the information available in each chunk is covered in custom corpus. To implement our modified RAG system, we construct an inverted index based on the embedding of generated questions. Each question is transformed into an embedding vector, which is then quantized to the nearest prototype to reduce the computational load and facilitate efficient retrieval. The quantization process is key to building the inverted index, which efficiently links each prototype to related text chunks for fast retrieval. We deploy our system on the open-source Meta-LLaMA3-8B-instruct model. Throughout this implementation, our focus has been on maintaining the integrity and trustworthiness of the responses achieved through our system.

When a user submits a query, the query is initially converted into a vector using the same encoding technique employed for the previously generated questions. This vector is then quantized to the closest prototype in the embedding space, determined through cosine similarity. An inverted index is utilized to efficiently retrieve all embedding vectors associated with this prototype. For each of these vectors, the corresponding original question is decoded that reveals the associated text chunk from our dataset. Each chunk is not only rich in content but is also associated with a source link, providing direct access to the original documents. These chunks correspond to the sections of documents that are most relevant to the user's query. Offering direct access to the sources fulfills two key objectives: a) it establishes a solid foundation for the trustworthiness of responses, and b) it encourages users to further explore and verify the information themselves to get a more interactive and reliable user experience. Instead of simply returning these chunks, our system uses them as context to generate a coherent response. This process leverages Meta-LLaMA3-8B-instruct model to produce an answer that is not only contextually aligned with the query but also substantiated by factual content from the original documents. It ensures the accuracy and relevance of the responses provided.

To evaluate the effectiveness of our dataset creation methodology and QuIM-RAG, we conduct a comprehensive assessment using the North Dakota State University (NDSU) website as a case study. The research question we are focusing on is:

Question 1: "How does creating a web-retrieval dataset in a specific way impact the accuracy of responses generated by an LLM-powered system?"

Question 2: "How does an advanced RAG model with a novel retrieval mechanism perform relative to a conventional RAG model?"

In response, we perform a thorough comparison of the performance between our LLM-powered modified RAG system and a traditional RAG system, employing both a conventional web retrieval dataset and our custom dataset. The results from our comparative analysis highlight significant and considerable improvements in accuracy with the use of the custom dataset. Moreover, we aim to validate the effectiveness of our novel retrieval mechanism in enhancing the precision and reliability of the overall RAG system.

II. RELATED WORK

The development of Large Language Models (LLMs) has significantly transformed Natural Language Processing (NLP) in their ability to process and generate human-like text. State-of-the-art models like OpenAI's GPT series, Google's PaLM, and Meta's LLAMA series primarily utilize the Transformer architecture, which underpins their sophisticated understanding and generation of language [13], [14] [15], [16]. These models showcase diverse architectural strategies, including exclusive use of decoders (as in GPT-2 and GPT-3), encoders (such as BERT and RoBERTa), or a blend of both in encoder-decoder frameworks (like BART), highlighting their versatility in approaching various linguistic tasks.

Providing accurate answers to a given question using LLMs has become one of the most explored research areas in the last couple of years [17], [18] [19], [20] [21]. The evaluation of Q&A research has significantly shifted towards domain-specific applications, particularly in complex fields such as law, medicine, and education [22]. While LLMs have made significant advances in generating human-like responses, they sometimes fall short in areas requiring specialized knowledge and are prone to producing inaccurate information, a phenomenon known as hallucination [23]. This limitation often arises because LLMs are typically trained on broad, general datasets that may not cover niche topics extensively. One way of overcoming this limitation is to use RAG for better performance in Q&A [24], [25]. Additionally, the introduction of Probably Asked Questions (PAQ) and the QA-pair retriever, RePAQ, highlights the evolving approaches to enhance LLMs' efficiency in Q&A, aiming to bridge the gap in knowledge coverage through QA-pairs [26].

In the context of RAG, it is crucial to efficiently retrieve relevant documents from the data source [11]. For that, data curation and preparation are important. RAG has been used for Open-Domain Question Answering (ODQA) using publicly available datasets that are easily accessible. However, few works have been done where custom datasets were required through web retrieval, content extraction, and segmentation [27], [28] [29], [30]. A major challenge with RAG is its tendency to hallucinate with large text volumes. Our study suggests creating a structured Q&A dataset from web text to enhance RAG's accuracy and reduce misinformation.

III. METHODOLOGY

Given a specific natural language question, the QA problem for a limited corpus is to identify and extract the most relevant and accurate answer from a predefined and constrained set of documents. The challenge lies in efficiently finding the most relevant answer that fits the question, taking into account the limited resources. Our key contribution is to pose finding the relevant answer as a matching process between potential questions that could have been asked for a document chunk with the actual user question. In this section, we first formalize the QA problem and then present our inverted question matching approach.

A. QA PROBLEM FOR LIMITED CORPUS

Given a corpus C consisting of a finite set of documents $\{D_1, D_2, \ldots, D_n\}$ and a natural language question Q, the task is to find the most relevant answer A from the corpus. Each document D_i in the corpus C contains a set of chunks (sentences) $\{S_{i1}, S_{i2}, \ldots, S_{im}\}$. We only focus on natural language queries that seek specific information contained within the corpus. The output is a text snippet response derived from the corpus, that directly addresses the question Q. A is a phrase generated from a combination of multiple pieces of information from the corpus. Hence the goal is to produce answer A such that A maximizes the relevance to Q within the constraints of C.

$$A = \arg\max_{A' \in C} \text{Relevance}(A', Q)$$

Here relevance is a scoring function that quantifies how well A' answers Q using context from corpus C. That might be computed using methods such as exactly matching expected keywords, using embeddings to measure the similarity between the question and potential answers, or ensuring that the answer fits logically with the surrounding text in the document. These natural solutions are either ineffective or prohibitive to implement. Our approach addresses this by generating hypothetical questions for each chunk S_i and finding the best match for the actual question to find the most likely chunk that answers the user query.

B. INVERTED QUESTION MATCHING TO FIND RELEVANT CHUNKS FOR ANSWER

Relevant chunks from various documents in the corpus can be combined to build the relevant informational context to answer the user's question. Here we describe our inverted index scheme for matching document chunks in embedding space.

1) CORPUS AND DOCUMENT STRUCTURE

The corpus C consists of a collection of documents $D_1, D_2, \ldots, D_n$. Each document D_i is composed of a set of chunks or sentences $S_1, S_2, \ldots, S_m$. These chunks represent the fundamental units of information within each document, and they serve as the basis for generating semantically meaningful questions that capture the content of the text.

2) QUESTION GENERATION FROM CHUNKS

For each chunk S_j within a document D_i, an instruction-following large language model (LLM) is employed to generate a set of questions $\{q_{ij1}, q_{ij2}, \ldots, q_{ijk}\}$. These questions are designed to encapsulate the key information or concepts contained in the chunk S_j. The process of question generation is crucial as it translates the raw text into a set of queries that can be later used for effective document

retrieval. Intuitively, these can be viewed as "frequently asked questions" for the given chunk.

3) EMBEDDING OF GENERATED QUESTIONS

Once the questions $\{q_{ij1}, q_{ij2}, \ldots, q_{ijk}\}$ are generated, each question q_{ijl} is transformed into an embedding vector $v_{ijl} \in \mathbb{R}^d$ using a pre-trained encoding scheme. These embedding vectors capture the semantic meaning of the questions in a high-dimensional space, enabling the system to compare and match questions based on their content.

4) QUANTIZATION OF EMBEDDINGS

Given the high dimensionality of the embedding space, direct comparison of all vectors would be computationally expensive. To mitigate this, each embedding vector v_{ijl} is quantized to the nearest prototype $p_l \in \{p_1, p_2, \ldots, p_k\}$. The quantization is performed by finding the prototype p_l that minimizes the cosine similarity distance to v_{ijl}:

$$p_l = \arg\min_p \text{CosineSimilarity}(v_{ijl}, p)$$

This quantization step reduces the complexity of the index and facilitates efficient matching of user queries.

5) CONSTRUCTION OF THE INVERTED INDEX

An inverted index I is built to map each prototype p_l to a set of embedding vectors v_{ijl} and their corresponding text chunks S_j. The index is constructed as follows:

$$I(p_l) = \{(v_{ijl}, S_j) \text{ for all } v_{ijl} \text{ quantized to } p_l\}$$

This index enables fast lookup of relevant text chunks based on the quantized embeddings.

6) QUERY PROCESSING AND MATCHING

When a user submits a query Q, it is first embedded into a vector V using the same encoding scheme applied to the generated questions. The vector V is then quantized to the nearest prototype p_l in the embedding space using cosine similarity:

$$p_l = \arg\min_p \text{CosineSimilarity}(V, p)$$

The inverted index I is then used to retrieve the set of embedding vectors v_{ijl} that were quantized to p_l.

7) RETRIEVAL OF RELEVANT TEXT CHUNKS

For each retrieved embedding vector v_{ijl}, the corresponding original question q_{ijl} is decoded, and the associated text chunk S_j is identified. These text chunks S_j represent the parts of the documents that are most relevant to the user query Q.

C. ANSWER GENERATION FOR USER QUERY

the relevant text chunks are returned to the large language model (LLM) to generate answer. However, in our scheme we use those chunk as the "context" for generating a coherent response that is directly based on the query and supported by factual information from the original document.

Algorithm 1 Inverted Index in Embedding Space With Question Generation and Matching

Input: Corpus $C = \{D_1, D_2, \ldots, D_n\}$, Prototypes $\{p_1, p_2, \ldots, p_k\}$
Output: Relevant text chunks for user query Q
function GenerateQuestions(S_i)
    Use the Instruction-following LLM to generate questions $\{q_{i1}, q_{i2}, \ldots, q_{ik}\}$ from chunk S_i
    return Questions $\{q_{i1}, q_{i2}, \ldots, q_{ik}\}$
end function
function EmbedQuestions($\{q_{i1}, \ldots, q_{ik}\}$)
    for each question q_{ij} **do**
        Compute embedding $v_{ij} \leftarrow \text{Encode}(q_{ij})$
        Append v_{ij} to the list of embeddings
    end for
    return Embeddings $\{v_{i1}, v_{i2}, \ldots, v_{ik}\}$
end function
function QuantizeEmbedding(v_{ij}, $\{p_1, \ldots, p_k\}$)
    return Prototype $p_l \leftarrow \arg\min_p \text{CosineSimilarity}(v_{ij}, p)$
end function
function BuildInvertedIndex(C, $\{p_1, \ldots, p_k\}$)
    Initialize an empty inverted index I
    for each document D_i in C **do**
        for each chunk S_j in D_i **do**
            Questions $\leftarrow$ GenerateQuestions(S_j)
            Embeddings $\leftarrow$ EmbedQuestions(Questions)
            for each embedding v_{ij} in Embeddings **do**
                Prototype $p_l \leftarrow$ QuantizeEmbedding(v_{ij}, $\{p_1, \ldots, p_k\}$)
                Update $I(p_l) \leftarrow I(p_l) \cup \{(v_{ij}, S_j)\}$
            end for
        end for
    end for
    return I
end function
function MatchQuery(Q, $\{p_1, \ldots, p_k\}$, I)
    Compute query embedding $V \leftarrow \text{Encode}(Q)$
    Find the nearest prototype $p_l \leftarrow \arg\min_p \text{CosineSimilarity}(V, p)$
    Retrieve relevant embeddings $E \leftarrow I(p_l)$
    for each $(v_{ij}, S_j$ in E **do**
        Decode v_{ij} to original question q_{ij}
        Append chunk S_j to the list of relevant texts
    end for
    return Relevant text chunks
end function

IV. EXPERIMENTAL SETUP

Our work started with detailed data collection directly from two primary sources within the North Dakota State University (NDSU) domain: the NDSU Career Advising (https://career-advising.ndsu.edu) and the NDSU Catalog (https://catalog.ndsu.edu) websites. These websites were chosen due to their comprehensive and versatile information. While we aimed to gather all available information from both sites, the Career Advising site predominantly offered insights into career guidance, job search strategies, assistance with resume tailoring, interview preparation, and details on club activities, including their missions, visions, and current engagements. Simultaneously, the Catalog website provided us with comprehensive information on academic aspects such as admissions, enrollments, detailed course descriptions, listings of required and elective courses for each academic

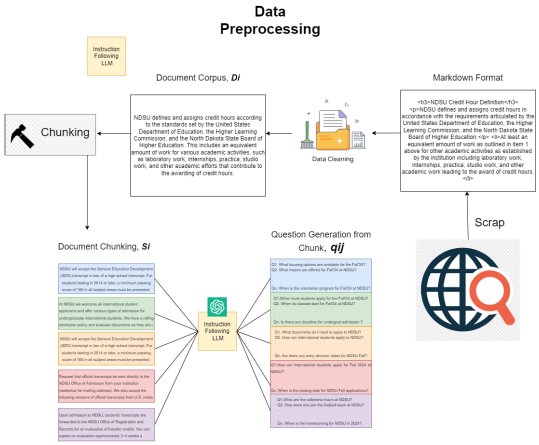


FIGURE 1. Overall Architecture of Corpus Preparation for Modified RAG.

program, graduation requirements, and academic rules and policies.

We employed advanced web scraping techniques, beginning with the main pages of NDSU-Career Advising (https://career-advising.ndsu.edu) and NDSU-Catalog (https://catalog.ndsu.edu). We then systematically navigated through their related sub-pages for comprehensive coverage of all available content on these sites. This methodology is structured into three main phases: Data Preparation, Retrieval, and Generation.

A. DATA PREPARATION

To create an accurate and complete dataset [Figure 1], we have constructed a customized web crawler integrating the BeautifulSoup and Scrapy frameworks. These two frameworks helped to traverse two primary sub-domains: NDSU Career Advising (https://career-advising.ndsu.edu) and NDSU Catalog (https://catalog.ndsu.edu). Starting from these parent links, the scraping process was designed to first identify and collect all associated page links reachable from these initial entry points. After collecting links, all HTML source code was pulled with the help of the crawler. Some data post-processing was done by removing the website header, footer, and unnecessary HTML tags to produce a cleaner version of the data. We also filtered out pages with less than 250 characters and those with a "404 page not found" title to ensure a relevant and high-quality dataset. The collection of all accessible links and their content offered a full insight into the website's resources and helped develop a broader dataset.

The next phase involved deploying an instruction following large language model (LLM) to construct a targeted data corpus, for which we utilized GPT 3.5-turbo-instruct provided by OpenAI. This model is designed to interpret and execute instructions seamlessly. We employed TikToken for chunking the data into 1000 tokens and creating overlapping chunks of 200 characters to ensure that the model comprehended the context of each chunk effectively. For each distinct chunk, a set of questions is generated to encapsulate the key information of the chunk. These questions are later used for effective context retrieval and help with accurate answer generation. The number of question-answer pairs generated for each context varies. This methodical approach enabled us to develop a customized dataset for our RAG system to ensure thorough coverage of each topic.

During the development of our data corpus, we placed great emphasis on ensuring that each set of questions is both accurate and relevant to the chunk. To achieve this, we implemented a detailed manual review process following the initial automated generation of questions. This review was conducted by two researchers who closely examined each set to confirm its factual accuracy and contextual relevance. During this review, we identified instances where the automated system generated questions that did not adequately address the chunk or were incomplete. We reprocessed these sections to resolve any inconsistencies and ensure that each set accurately mirrored the data it was derived from. We also eliminated duplicates and irrelevant questions to maintain the integrity and value of the corpus. Furthermore, we carefully check for semantic and syntactic errors to ensure information correctness and grammatical accuracy. This enhanced the overall reliability of how we prepare the dataset.

TABLE 1. Summary of the NDSU catalog and career advising datasets.

Metric	NDSU Catalog Dataset	NDSU Career Advising
No. of Links Scraped	288	399
No. of Questions	9,027	13,582
No. of chunks	296	435

Table 1 provides an overview of the dataset used in the paper. The data was obtained by scraping 687 links associated with North Dakota State University's (NDSU) career-advising and catalog resources. The scraped context was then formed into manageable chunks with set of questions and links using GPT-3.5-turbo-instruct model. This resulted in 9,027 questions from the NDSU Catalog dataset and 13,582 questions from the NDSU Career Advising dataset, resulting in a total of 22,609 questions.

B. GROUND TRUTH PREPARATION

We prepared ground truth data to ensure high accuracy and relevance for evaluating our RAG system. The process began by selecting relevant content from the university website, which served as potential sources of information. We extracted this content using automated web scraping tools to preserve all essential details. we then manually curated this content to verify its factual accuracy and direct relevance to potential user queries. Based on this verified content, we created question-answer pairs that accurately reflect the information available on the website. We ensured that each pair was complete and directly linked to its original source. This prepared ground truth helps to effectively assess the performance of our RAG system.

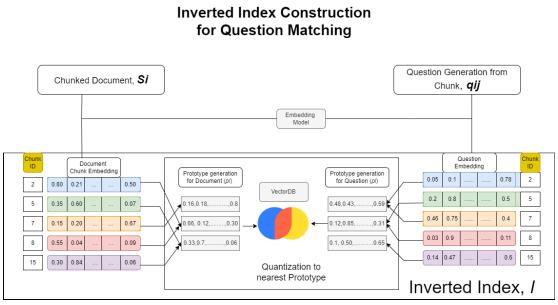


FIGURE 2. Illustration of Inverted Index Construction for Question Matching.

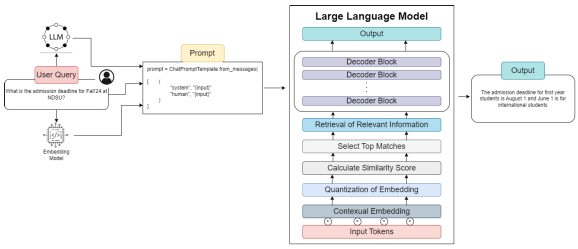


FIGURE 3. Overall Retrieval and Generation Architecture for RAG.

C. INVERTED INDEX CONTRUCTION FOR QUESTION MATCHING

After constructing the dataset corpus, the next step involves preparing an inverted index to facilitate efficient question matching (Figure 2). Once the questions are generated, we use an embedding vector to capture the semantic meaning of each chunk and data corpus. We employed a pretrained embedding model from HuggingFace (model name: BAAI/bge-large-en-v1.5) for its high performance on the Multi-Task Evaluation Benchmark (MTEB) [31]. This model utilizes a flag embedding technique, which can excel in the semantic search and retrieval capability of any LLM. The vector representations for both documents and questions are subsequently quantized into the nearest prototype vectors in a high-dimensional space. We use chromaDb as a vector database for the quantization process that stores prototype vectors for both document chunks and generated questions. The vectors are represented in a format where each prototype vector is derived by minimizing the cosine similarity distance between the chunk or question embedding and available prototypes to effectively reduce the computational complexity of direct vector comparisons. These quantized embeddings are then formed in an inverted index, which maps each prototype vector to the corresponding text chunks and their derived questions. This inverted index serves as the backbone of the retrieval process. To efficiently retrieve questions and corresponding chunks, we use chunk ids in chunk vectorDB and question vectorDB.

D. RETRIEVAL

The retrieval system is designed to efficiently match user queries with the most relevant information within a comprehensive knowledge base. When a user submits a query Q, our system initiates by transforming Q into an embedding vector using the same embedding process that applied to data corpus. Then, from the inverted index, it matches for the closest semantic question from data corpus by similarity search using cosine similarity. It selects the top 3 questions ($k = 3$) that best align semantically with the query. Then the system retrieved the associated chunks related to these questions and decode and return these as context to the llAma3-7b-instruct.

E. GENERATION

The final response to the user is generated by an open-source large language model (LLM) from Hugging Face named Llama3-8b-instruct. This is the latest addition to the Llama series while we were writing the paper, with significant advancements in AI capability. We have used the instruct-tuned version for its powerful conversational and chat capability. Llama3 supports a context length of 8,000 tokens, which allows for more extended interactions and more complex input handling compared to many previous models. Longer context capacity ensures that the model can consider a wider range of information to deliver the most relevant and coherent responses.

In the generation process, the model retrieves the most relevant context from the retrieval model. Then it integrates the information seamlessly using a custom prompt (Figure 4). Since the Llama3 model is trained on a vast database from diverse sources, it has a broad understanding of generating human-like text. This training enables the model to effectively utilize the retrieved information $\{a_i\}$ to generate a response that is not only relevant but also contextually coherent. The final response is then delivered back to the user as output.

F. PROMPT

To refine the capabilities of RAG, developing a comprehensive dataset is essential. This dataset is crucial for training the RAG system to deliver precise and contextually relevant responses. Acknowledging the necessity to encompass the breadth of information within our dataset, we formulated a custom prompting strategy. This critical step ensures that the resultant dataset of question-and-answer (Q&A) pairs is not only comprehensive but also precise and reflective of the diverse content within the source texts.

Our goal is to achieve comprehensive coverage of the chunk and ensure the diversity and accuracy of generated set of questions. The designed prompt [figure 4] explicitly instructs the model to generate a set of questions that cover all the key information of each chunk. The prompt directs the model to avoid redundancy, ensuring that each question is unique and contextually relevant to the text. This approach helps prevent the generation of overlapping questions and ensures that each query contributes distinct value to understanding the chunk's content.

Building upon this foundation, it is imperative to address how our RAG model manages queries that extend beyond the confines of its knowledge base. To tackle different types of questions effectively and prevent the provision of inaccurate responses to out-of-domain (OOD) inquiries, another prompting strategy was implemented for the RAG

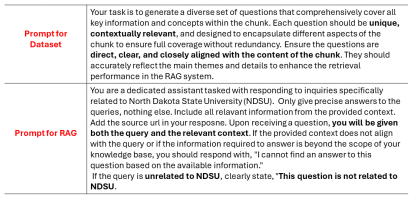


FIGURE 4. The upper section details a prompt designed for creating a custom dataset, focusing on generating set of questions for each chunk. The lower section outlines a prompt for a RAG system, emphasizing accuracy and directive responses based on the dataset, with instructions on how to handle queries that extend beyond available data.

model itself. In instances where a query seeks information absent from the text segments known to the model, the model is prompted to clearly state its inability to provide a relevant answer. This ensures the model explicitly knows its boundaries and helps prevent the provision of false or invented responses.

G. COMPARISON TO TRADITIONAL RAG SYSTEM

Traditional RAG systems typically follow a straightforward ''Retrieve-Read'' methodology [11]. These systems begin with an indexing phase where data is segmented into vectorized chunks. These chunks are then retrieved in response to user queries based on their semantic similarity. However, these systems often struggle with information overload and accuracy, leading to responses that may not always be contextually appropriate or factually correct. This affects the overall quality of the responses they produce. In contrast, our advanced RAG system introduces substantial improvements in both the retrieval and generation phases to address these limitations. Our retrieval system is designed to match user queries with the most relevant information within a comprehensively curated knowledge base. By employing an enhanced semantic chunking method, our system segments data into coherent units that encapsulate complete thematic entities. The retrieval process in our advanced RAG system utilizes state-of-the-art embedding models that improve the traditional vectorization methods. It features a refined retrieval system that uses state-of-the-art embedding models for better semantic matching and segments data into coherent units to make the retrieved information highly relevant to user queries. The generation phase is powered by the Llama3-8b-instruct model from Hugging Face, which supports complex inputs and provides coherent outputs over extended conversational turns. Additionally, we've implemented a custom prompting strategy to ensure the generated Q&A pairs are detailed and accurate and significantly minimizing issues like redundancy and hallucination commonly found in traditional RAG systems.

V. EVALUATION

To evaluate the performance of our QuIM-RAG model, we employ two evaluation frameworks, BERTScore [32] and RAGAS [33]. These frameworks will provide a detailed quantitative analysis that focuses on the semantic accuracy and relevance of the models response. We use BERTScore and RAGAS instead of more commonly used metrics such as BLEU [34], METEOR [35], and ROUGE [36] which don't quite match our evaluation needs. BLEU and METEOR are traditionally applied to machine translation tasks, and ROUGE is tailored for text summarizing assessments. Given that our model operates within a RAG-based question-answering context, these conventional metrics fall short in accurately measuring its performance. This limitation has led us to select evaluation methods that are more appropriate and tailored to our model's specific needs.

A. BERTSCORE

BERTScore is used to evaluate the semantic quality of text by measuring the overlap between model-generated outputs and reference texts [32]. Unlike traditional metrics that rely on n-gram overlaps, BERTScore utilizes contextual embedding to capture deeper semantic meanings to it. It is highly effective for tasks where linguistic precision is crucial. BERTScore operates by first transforming both the reference, $\mathbf{x} = (x_1, x_2, \ldots, x_n)$, and candidate, $\hat{\mathbf{x}} = (\hat{x}_1, \hat{x}_2, \ldots, \hat{x}_m)$ texts into contextual embedding using models like BERT or RoBERTa[].

$$\text{BERT}((x_1, x_2, \ldots, x_n)) = (X_1, X_2, \ldots, X_n)$$
$$\text{BERT}((\hat{x}_1, \hat{x}_2, \ldots, \hat{x}_m)) = (\hat{X}_1, \hat{X}_2, \ldots, \hat{X}_m)$$

These embeddings integrate the surrounding context of words to provide a robust representation of textual meaning. The core of BERTScore's evaluation method is the pairwise cosine similarity calculation between tokens of the reference text and the candidate text. For each token in the reference, it identifies the token in the candidate text that has the highest cosine similarity and vice versa.

$$\text{similarity}(X_i, \hat{X}_j) = \frac{X_i^T \hat{X}_j}{\|X_i\| \|\hat{X}_j\|}$$

To evaluate the quality of text generation, three distinct metrics are used: BERT-Precision, BERT-Recall, and BERT-F1.

BERT-Precision(BERT-P): This metric quantifies the semantic similarity between each token in the generated text and the nearest equivalent token in the reference text. By employing greedy matching, BERT-P optimizes the similarity scores to effectively handle the linguistic variability of semantic words that are interchangeable without changing the overall meaning. Greedy matching is critical in the language domain because multiple words may hold similar meanings to the ground truth, and the words of sentences can be structured in various ways without changing identical semantic content.

$$P_{\text{BERT}} = \frac{1}{|\hat{\mathbf{x}}|} \sum_{\hat{x}_j \in \hat{\mathbf{x}}} \max_{x_i \in \mathbf{x}} \left\{ \text{cosine similarity}(\hat{X}_i^T, X_j) \right\}_{\text{greedy matching}}$$

BERT-Recall (BERT-R): It measures the model's completeness by comparing each token of reference text with generated text that has highest cosine similarity. The recall

score is computed by averaging these maximum similarity scores.

$$R_{\text{BERT}} = \frac{1}{|\mathbf{x}|} \sum_{x_i \in \mathbf{x}} \max_{\hat{x}_j \in \hat{\mathbf{x}}} \left\{ \text{cosine similarity}(X_i^T, \hat{X}_j) \right\}_{\text{greedy matching}}$$

BERT-F1 Score (BERT F1): Combining the BERT-P and Bert-R, the F1 score provides a holistic measure of text generation quality by balancing both the breadth and depth of semantic content captured in the generated text.

$$F_{\text{BERT}} = 2 \times \frac{P_{\text{BERT}} \times R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}}$$

B. RETRIEVAL-AUGMENTED GENERATION ASSESSMENT (RAGAS)

RAGAS [33] is designed to evaluate the effectiveness of RAG systems that combine retrieval mechanisms with LLMs to provide accurate information. It evaluates these systems by focusing on the precision of context retrieval, the reliability of the generated content, and the overall quality of the text output. Importantly, RAGAS is constructed in such a way that it eliminates the need for human annotations in the evaluation process. Since Ragas is integrated with popular frameworks like LangChain and LLama-index, developers can conveniently utilize and assess their systems using this framework.

1) EVALUATION METRICS

RAGAS evaluates three primary aspects of RAG architectures:

- **Faithfulness:** This metric evaluates how well the retrieval system grounds the generated responses in the provided context. It involves extracting statements from the generated answers and verifying each statement against the retrieved context using a validation function to determine if the context supports the statement.

$$\text{Faithfulness} = \frac{\text{Number of verified claims}}{\text{Total claims made}}$$

- **Answer Relevance:** This metric evaluates the appropriateness of the generated answers to address the posed questions. It is measured by generating potential questions from the answers using an LLM and calculating the cosine similarity of these questions to the original question. The relevance score is the average of these similarities, which indicates the directness and appropriateness of the answers.

$$AR = \frac{1}{n} \sum_{i=1}^{n} \text{sim}(q, q_i)$$

  where q_i are questions generated based on the answer and sim is the cosine similarity between embeddings of the original question q and q_i.
- **Context Relevance:** This metric determines whether the retrieved context contains primarily relevant information needed to answer the question. The process involves extracting sentences from the context that are necessary to answer the question and then calculating the ratio of these extracted sentences to the total number of sentences in the context.

$$CR = \frac{\text{Number of relevant sentences extracted}}{\text{Total sentences in the context}}$$

VI. RESULT

The efficiency of our RAG system are demonstrated in Figure 5, which offers a detailed comparison between responses generated from traditional and custom datasets when queried about the location and contact information of the North Dakota State University (NDSU) Career and Advising Center. As shown in figure 5, when a user submits a query, it is first converted into a query vector. The encoded query vector is then used to compute similarity scores against a database of question vectors, which represent pre-stored questions associated with various chunks of information. The system identifies the top k matches (in this case, k=3), which are the questions most semantically related to the user query based on their similarity scores. For each of the top matching question vectors, the system retrieves the associated text chunks from the dataset. In this scenario, question 1 and 2 comes from the same chunk and question 3 comes from another chunk. These chunks contain the detailed information corresponding to each question. The retrieved chunks serve as context for the base LLM Meta-LLaMA3-8B-instruct to process this context along with the user query to generate a coherent and detailed answer that addresses the user's specific needs. This approach directly addressed the query with precise information, including the phone number, mailing address, and email contact for the career and advising center. Additionally, it provides the source link for users who wish to further explore or verify the information. The traditional dataset response is also begins with similar query processing steps. However, when querying for contact information, the traditional dataset yields the details of an Assistant Director, which, while correct, may not align with the general inquiry intentions of users seeking contact information. The custom dataset and QuIM-RAG help the LLM to understand users' specific needs and avoid providing excessively detailed or irrelevant information, which is a common issue with the traditional RAG system. The comparison highlights that the QuIM-RAG with custom dataset provides more targeted and relevant response to the user.

The efficacy of the RAG system was evaluated across a spectrum of metrics to ascertain the impact of employing traditional versus custom datasets on performance. Table 2 presents a comparative analysis of these metrics for both Traditional RAG and our novel QuIM-RAG systems. For the traditional RAG using traditional datasets, faithfulness scores reflected a moderate accuracy of 0.69, while the implementation with custom datasets showed an improved score of 0.72. The QuIM-RAG system, however, demonstrated superior faithfulness, particularly with custom datasets, achieving a perfect score of 1.00. This trend was mirrored across other

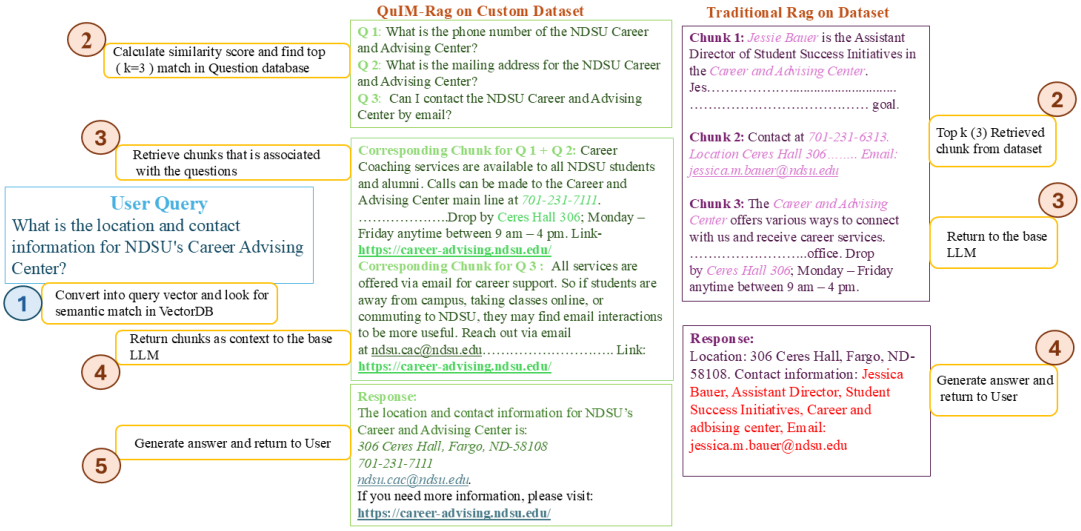


FIGURE 5. Workflow of QuIM-RAG system and Traditional RAG system for User Query Processing.

TABLE 2. Comparison of performance metrics for traditional RAG and our novel QuIM-RAG using traditional and custom datasets.

Evaluation Matrix	Traditional RAG		QuIM-RAG	
	Traditional	Custom	Traditional	Custom
Faithfulness	0.69	0.72	0.91	1.00
Answer Relevancy	0.79	0.82	0.93	0.99
Context Precision	0.45	0.69	0.82	0.92
Context Recall	0.39	0.45	0.60	0.74
Harmfulness	0	0	0	0
BERTScore P.	0.32	0.37	0.55	0.63
BERTScore R.	0.29	0.35	0.63	0.71
F1 Score	0.31	0.36	0.59	0.0.67

metrics, including Answer Relevancy and Context Precision, with the QuIM-RAG system outperforming the Traditional RAG, underscoring the benefits of custom data in enhancing system accuracy. The metrics of Context Recall and BERTScore precision and recall also exhibited significant improvements when transitioning from traditional to custom datasets within both RAG systems. Notably, the Traditional RAG system experienced a Context Recall increase from 0.39 to 0.45 with custom datasets, whereas our QuIM-RAG's recall improved from 0.60 to 0.74, suggesting a better ability to retrieve relevant information when utilizing custom data. A consistent score of zero for Harmfulness across all models indicates a successful avoidance of generating harmful content. This is a critical aspect, highlighting the systems' reliability and the effectiveness of underlying filtering mechanisms.

BERTScores further support the findings, with QuIM-RAG system achieving a precision of 0.63 and a recall of 0.71 using custom datasets, in comparison to 0.32 and 0.29, respectively, with the traditional RAG on traditional data. The F1 Score, a harmonic mean of precision and recall, showcases a conclusive elevation from 0.31 in the traditional RAG with traditional data to 0.67 in the QuIM-RAG with custom data. This quantifiable advancement validates the integration of custom datasets as a substantial enhancement to the RAG system's performance.

VII. CONCLUSION

This paper addressed challenges of using large language models (LLM) for domain-specific question-answering. By integrating an advanced retrieval-augmented generation (RAG) system and a methodological approach to data preparation has enhance the quality of responses generated by these systems. Creating a custom dataset specifically designed for the domain in question has been key in reducing common problems like information dilution and hallucination that often seen in traditional RAG systems when they handle large amounts of unstructured data. Our evaluations show that our novel QuIM-RAG system, leveraging a custom dataset and Llama-3-8b-instruct, improves the accuracy and relevance of its responses. It performs much better than the baseline dataset, which is made from raw web data. Additionally, the responses include source links with the user's query offers further opportunities for user to seek more information and verify the details provided. Given that university websites frequently update their content every semester, we are planning to design a content retrieval mechanism that updates its corpora every four months. This will ensure that any newly added content is incorporated into the system to keep the information up-to-date and relevant for users. Looking ahead, our future goal is to conduct a comprehensive user study to assess user satisfaction and system usability. The insights gained from this study will help us refine the system further to ensure that it continues to meet user needs effectively while remaining adaptable to evolving content.

ACKNOWLEDGMENT

These resources, including computational infrastructure and essential support services, were invaluable for the processing and analysis tasks involved in their research. The authors sincerely thank CCAST for their assistance.

REFERENCES

[1] T. B. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2020, pp. 1877–1901.
[2] F. Yang, P. Zhao, Z. Wang, L. Wang, J. Zhang, M. Garg, Q. Lin, S. Rajmohan, and D. Zhang, "Empower large language model to perform better on industrial domain-specific question answering," 2023, *arXiv:2305.11541*.
[3] N. Kandpal, H. Deng, A. Roberts, E. Wallace, and C. Raffel, "Large language models struggle to learn long-tail knowledge," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 15696–15707.
[4] O. Ovadia, M. Brief, M. Mishaeli, and O. Elisha, "Fine-tuning or retrieval? Comparing knowledge injection in LLMs," 2023, *arXiv:2312.05934*.
[5] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proc. Nat. Acad. Sci. USA*, vol. 114, no. 13, pp. 3521–3526, 2017.
[6] I. J. Goodfellow, M. Mirza, D. Xiao, A. Courville, and Y. Bengio, "An empirical investigation of catastrophic forgetting in gradient-based neural networks," 2013, *arXiv:1312.6211*.
[7] S. Chen, Y. Hou, Y. Cui, W. Che, T. Liu, and X. Yu, "Recall and learn: Fine-tuning deep pretrained language models with less forgetting," 2020, *arXiv:2004.12651*.
[8] B. Saha and U. Saha, "Enhancing international graduate Student experience through AI-driven support systems: A LLM and RAG-based approach," in *Proc. Int. Conf. Data Sci. Its Appl. (ICoDSA)*, Jul. 2024, pp. 300–304.
[9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2020, pp. 9459–9474.
[10] A. Neelakantan et al., "Text and code embeddings by contrastive pre-training," 2022, *arXiv:2201.10005*.
[11] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," 2023, *arXiv:2312.10997*.
[12] E. M. Bender, T. Gebru, A. Mcmillan-Major, and S. Shmitchell, "On the dangers of stochastic parrots: Can language models be too big?" in *Proc. ACM Conf. Fairness, Accountability, Transparency*, Mar. 2021, pp. 610–623.
[13] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
[14] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, and P. Schuh, "PaLM: Scaling language modeling with pathways," *J. Mach. Learn. Res.*, vol. 24, no. 240, pp. 1–113, 2023.
[15] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
[16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–11.
[17] M. U. Hadi, Q. A. Tashi, R. Qureshi, A. Shah, A. Muneer, M. Irfan, A. Zafar, M. B. Shaikh, N. Akhtar, J. Wu, and S. Mirjalil, "A survey on large language models: Applications, challenges, limitations, and practical usage," in *Proc. Authorea Preprints*, 2023, pp. 1–31.
[18] S. Huo, N. Arabzadeh, and C. L. A. Clarke, "Retrieving supporting evidence for LLMs generated answers," 2023, *arXiv:2306.13781*.
[19] L. Fan, L. Li, Z. Ma, S. Lee, H. Yu, and L. Hemphill, "A bibliometric review of large language models research from 2017 to 2023," 2023, *arXiv:2304.02020*.
[20] V. Bolotova, V. Blinov, F. Scholer, W. B. Croft, and M. Sanderson, "A non-factoid question-answering taxonomy," in *Proc. 45th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2022, pp. 1196–1207.
[21] A. Rogers, M. Gardner, and I. Augenstein, "QA dataset explosion: A taxonomy of NLP resources for question answering and reading comprehension," *ACM Comput. Surveys*, vol. 55, no. 10, pp. 1–45, Oct. 2023.
[22] S. Ruder and A. Sil, "Multi-domain multilingual question answering," in *Proc. Conf. Empirical Methods Natural Lang. Process., Tutorial Abstr.*, 2021, pp. 17–21. [Online]. Available: https://api.semanticscholar.org/CorpusID:245289877
[23] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Comput. Surveys*, vol. 55, no. 12, pp. 1–38, Dec. 2023.
[24] G. Mialon, R. Dessì, M. Lomeli, C. Nalmpantis, R. Pasunuru, R. Raileanu, B. Rozière, T. Schick, J. Dwivedi-Yu, A. Celikyilmaz, E. Grave, Y. LeCun, and T. Scialom, "Augmented language models: A survey," 2023, *arXiv:2302.07842*.
[25] A. Lazaridou, E. Gribovskaya, W. Stokowiec, and N. Grigorev, "Internet-augmented language models through few-shot prompting for open-domain question answering," 2022, *arXiv:2203.05115*.
[26] P. Lewis, Y. Wu, L. Liu, P. Minervini, H. Küttler, A. Piktus, P. Stenetorp, and S. Riedel, "PAQ: 65 million probably-asked questions and what you can do with them," *Trans. Assoc. Comput. Linguistics*, vol. 9, pp. 1098–1115, Oct. 2021.
[27] V. Sharma and V. Raman, "A reliable knowledge processing framework for combustion science using foundation models," 2023, *arXiv:2401.00544*.
[28] I. Siragusa and R. Pirrone, "Conditioning chat-gpt for information retrieval: The unipa-gpt case study," in *Proc. 7th Workshop Natural Lang. Artif. Intell. (NL4AI) Co-Located 22th Int. Conf. Italian Assoc. Artif. Intell. (AI IA)*, 2023, pp. 1–11.
[29] M. Kulkarni, P. Tangarajan, K. Kim, and A. Trivedi, "Reinforcement learning for optimizing RAG for domain chatbots," 2024, *arXiv:2401.06800*.
[30] S. Campese, I. Lauriola, and A. Moschitti, "QUADRo: Dataset and models for QUestion-answer database retrieval," 2023, *arXiv:2304.01003*.
[31] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive text embedding benchmark," 2022, *arXiv:2210.07316*.
[32] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," 2019, *arXiv:1904.09675*.
[33] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," 2023, *arXiv:2309.15217*.
[34] S. Kumar, A. Solanki, and N. Z. Jhanjhi, "Rouge-SS: A new Rouge variant for the evaluation of text summarization," 2024.
[35] S. Banerjee and A. Lavie, "METEOR: An automatic metric for MT evaluation with improved correlation with human judgments," in *Proc. ACL Workshop Intrinsic Extrinsic Eval. Measures Mach. Transl. Summarization*, Jun. 2005, pp. 65–72.
[36] M. Barbella and G. Tortora, *Rouge Metric Evaluation for Text Summarization Techniques*, Standard SSRN 4120317, 2022.



BINITA SAHA received the B.Sc. degree in computer science and engineering from BRAC University, Dhaka, Bangladesh, in 2017, and the M.Sc. degree in computer science from North Dakota State University (NDSU), Fargo, ND, USA, in 2021, where she is currently pursuing the Ph.D. degree with a focus on machine learning and cybersecurity.



UTSHA SAHA (Graduate Student Member, IEEE) received the B.Sc. degree in computer science and engineering from American International University-Bangladesh, in 2019. He is currently pursuing the M.Sc. degree in computer science with North Dakota State University (NDSU), Fargo, ND, USA, with a specialization in machine learning and bioinformatics. In addition to his studies, he works as a Software Developer with the NDSU Agricultural Data Analytics Unit, where he focuses on developing data-driven applications.



MUHAMMAD ZUBAIR MALIK received the master's degree in computer engineering from The University of Texas at Austin, in 2007, and the Ph.D. degree in software engineering, in 2009. He is currently an Assistant Professor with the Department of Computer Science, North Dakota State University. Before starting his career as a Professor, he was a Researcher with Carnegie Mellon University.